

The rectangular face-jet decomposition

Claude, with Daniel Murfet

2026-09-07

Abstract

For face jets $a_i(v, s)$, $b_j(u, s)$ and corner jets $c_{ij}(s)$ the rectangular remainder is

$$R = \Phi - \sum_{i < M_1} u^i a_i(v, s) - \sum_{j < M_2} v^j b_j(u, s) + \sum_{i < M_1} \sum_{j < M_2} c_{ij}(s) u^i v^j,$$

and the block integral of Φ is the corresponding sum of block integrals (`twoDamp_rect`). Monomial weights are absorbed into the exponents, so that a face term $u^i a_i(v, s)$ is the one-variable amplitude a_i with exponents $(h_1 + i, h_2)$ and a corner term is the constant amplitude c_{ij} with exponents $(h_1 + i, h_2 + j)$. Each piece is now exactly the input of an existing theorem: the face terms of unit 104, the corner terms of unit 104 with zero remainder, the remainder of unit 105. This is the algebraic skeleton of Astra #4(b)'s cutoff-indexed $d = 2$ theorem; the next unit assembles it. Zero sorry/axiom.

1 The things formalised

```
noncomputable def rectRem (ϕ : → → → ) (a b : → → → ) (c : → → → )
  (M M : ) (u v s : ) : :=
  ϕ u v s - ∑ i Finset.range M, u ^ i * a i v s - ∑ j Finset.range M, v ^ j * b j u s
  + ∑ i Finset.range M, ∑ j Finset.range M, c i j s * (u ^ i * v ^ j)
theorem rectRem_continuous ...
theorem twoDIntegrand_rectRem ... -- linearity of the product-form integrand
theorem twoDamp_rect ( b N : ) (h h k k : ) (ϕ : → → → ) (a b j : → → → )
  (c : → → → ) (M M : ) (hϕ : Continuous fun x : × × => ϕ x.1 x.2.1 x.2.2)
  (ha : i, Continuous (Function.uncurry (a i)))
  (hb : j, Continuous (Function.uncurry (b j j))) (hc : i j, Continuous (c i j)) :
twoDamp b N h h k k ϕ
  = ( ∑ i Finset.range M, twoDamp b N h h k k (fun u v s => u ^ i * a i v s))
  + ( ∑ j Finset.range M, twoDamp b N h h k k (fun u v s => v ^ j * b j j u s))
  - ( ∑ i Finset.range M, ∑ j Finset.range M,
    twoDamp b N h h k k (fun u v s => c i j s * (u ^ i * v ^ j)))
  + twoDamp b N h h k k (rectRem ϕ a b j c M M)
theorem twoDamp_pow_absorb ( b N : ) (h h k k i : ) (Ψ : → → → ) :
twoDamp b N h h k k (fun u v s => u ^ i * Ψ v s)
  = twoDamp b N (h + i) h k k (fun _ v s => Ψ v s)
theorem twoDamp_pow_absorb' ... :
twoDamp b N h h k k (fun u v s => v ^ j * Ψ u s)
  = twoDamp b N h (h + j) k k (fun u _ s => Ψ u s)
theorem twoDamp_corner ( b N : ) (h h k k i j : ) (c : → → → ) :
twoDamp b N h h k k (fun u v s => c s * (u ^ i * v ^ j))
  = twoDamp b N (h + i) (h + j) k k (fun _ _ s => c s)
```

2 Mathematics

The product-form integrand $u^{h_1}v^{h_2}e^{-\beta s^2}\Phi(u,v,s)$ is linear in Φ , so the identity is a matter of integrability: every piece is continuous on the closed box, hence integrable for the product measure, and `integral_finsetSum/integral_sub/integral_add` distribute the integral over the finite sums. The absorption lemmas are pointwise: $u^{h_1} \cdot u^i = u^{h_1+i}$ inside the outer integral (after pulling u^i out of the inner one) and $v^{h_2} \cdot v^j = v^{h_2+j}$ inside the inner one.

3 Paper-facing meaning

Under Astra #4(b)'s hypotheses (compatible face jets a_i, b_j with Taylor data c_{ij} and a mixed remainder $|R| \leq Hu^{M_1}v^{M_2}$), the $d = 2$ block is now a finite sum of terms each of whose N -expansion is known: face terms via `face_transfer_bound` (exponents $p + m/k_2$ with a logarithm at the collisions $i/k_1 = m/k_2$), corner monomials via the same theorem with zero remainder, and the mixed remainder via `twoDAmp_env_isBig0` at order $N^{-\min(p+M_1/k_1, p+M_2/k_2)}(1 + \log N)$.

4 Lean notes

- `simp_rw` with a lemma whose explicit arguments are placeholders (`twoDAmp_eq_prod ... (hA _)`) cannot instantiate the metavariables; rewrite the sums termwise through `Finset.sum_congr` in named `haves` instead.
- `Integrable.sub` yields a `Pi.sub`; ascribe the single-lambda type before `← integral_sub` (the unit-77 gotcha again).
- After `congr 1` on `a * ... = a * ...` the inner `setIntegral_congr_fun measurableSet_Ioc` left `ClosedIicTopology ?m` stuck; state the inner-integral identity as a `have` with explicit integrands and `rw` it.
- With both `Set` and `Finset` open, `Ioc/range` are ambiguous; open `Set` only and write `Finset.range`.